

BleepKit — Complete Build Specification

Paste this entire document as your first message to the coding agent (Claude Code, Copilot, Cursor). It is self-contained and written to be executed top to bottom without follow-up questions.

PART I — AGENT OPERATING RULES

You are a senior iOS engineer with deep AVFoundation and Core Animation experience. You are building a complete, shippable iOS application. Follow these rules for the entire session.

1. **Build in phases.** Nine phases are defined in Part VI. Each ends with a Verification Gate. Stop at each gate, compile, confirm the gate passes, and report before starting the next phase. Do not write Phase 6 code while Phase 3 is unverified.
2. **No placeholders.** Every function body is real, working code. No `// TODO`, no `fatalError("unimplemented")`, no stubbed returns, no `return []` standing in for logic, no comments describing code you did not write.
3. **Verify every API before you use it.** If you are not certain a symbol exists with the exact signature you are about to write, stop and check the SDK headers or Apple's documentation. Do not invent method names. Do not guess parameter labels or argument order. Part III pins the signatures for every non-obvious API in this project — use those verbatim.
4. **Availability-gate everything.** Any symbol newer than the deployment target sits behind `if #available(...)` with a working fallback. No `@available` attribute on a type that gets instantiated unconditionally.
5. **Compile after every file.** Fix warnings, not just errors. Swift 6 strict concurrency must be clean — zero data-race diagnostics, zero `Sendable` warnings.
6. **If something cannot work as specified, say so and propose the alternative.** Do not silently substitute a different approach and continue. Flag the conflict, state why, propose the fix, wait.
7. **Never add a dependency.** No Swift Package Manager entries, no CocoaPods, no Carthage, no vendored source from GitHub. First-party Apple frameworks only. If you believe a task requires a library, you are wrong — solve it with the SDK or flag it under

rule 6.

8. **Never add a network call.** No `URLSession`, no `NWConnection`, no analytics, no crash reporting, no remote config. The app must function identically in Airplane Mode. Do not add any networking entitlement to the app.

PART II — PRODUCT DEFINITION

What it is

BleepKit is an iOS app for short-form video creators. It takes a vertical video — typically an Instagram Reel — transcribes it entirely on-device, generates burned-in animated captions, automatically detects profanity, and censors it in three synchronized layers.

The three censor layers

Layer	Behavior
Audio	Mute the original audio across the profane word's time range; layer a synthesized beep tone over it
Caption text	Replace the word in the burned-in caption with a censor treatment the user selects (asterisks, black bar, image, or emoji)
Video overlay	Optionally composite a user-chosen image over the video frame for the exact duration of the word

All three are driven by the same merged time ranges, so audio and video censoring are frame-aligned. This is a hard requirement — a beep that starts 3 frames before the black bar looks broken.

Output

A new video file, 1080×1920, H.264 or HEVC in an MP4 container, saved to the user's Photo Library in a "BleepKit" album, ready to upload to Instagram without further editing.

Hard constraints

- **Apple frameworks only.** Zero third-party code.

- **Zero network calls.** Fully functional in Airplane Mode.
- **Free to run.** No paid APIs, no subscriptions, no per-call costs. The user signs with a personal Apple Developer certificate.
- **On-device processing.** No audio or video leaves the device under any circumstance.

Permitted frameworks

`SwiftUI`, `SwiftData`, `AVFoundation`, `AVKit`, `Speech`, `Photos`, `PhotosUI`, `CoreMedia`, `CoreImage`, `CoreAudio`, `AudioToolbox`, `CoreText`, `CoreGraphics`, `QuartzCore`, `UniformTypeIdentifiers`, `Observation`, `OSLog`.

Platform targets

- Deployment target: **iOS 18.0**
- Swift language mode: **Swift 6** (strict concurrency)
- Devices: iPhone only, portrait only
- Xcode: latest stable

PART III — VERIFIED API CONTRACTS

These are the APIs most commonly hallucinated or misused. Use these signatures exactly as written.

3.1 SpeechTranscriber — primary transcription path (iOS 26+)

```
import Speech

// @available(iOS 26.0, macOS 26.0, *)
let transcriber = SpeechTranscriber(
    locale: Locale.current,
    transcriptionOptions: [],
    reportingOptions: [], // .volatileResults only for live stream
    attributeOptions: [.audioTimeRange] // MANDATORY — this is the source of wor
)

let analyzer = try SpeechAnalyzer(modules: [transcriber])
```

Facts you must build around:

- `attributeOptions: [.audioTimeRange]` is not optional for this app. Without it you receive text with no timestamps, and the entire product is impossible to build. If you find yourself with tokens that have no time range, this is the cause.
- Results arrive as `AttributedString`, not as a segments array. Iterate `result.text.runs` and read each run's `audioTimeRange` attribute, which is a `CMTimeRange`. That range is your word timing. Do not look for `bestTranscription.segments` — that belongs to the older API in §3.2.
- The language model is **not bundled with your app binary**. Before the first analysis, check `SpeechTranscriber.supportedLocales` and `SpeechTranscriber.installedLocales`. If the locale is supported but not installed, obtain an installation request from `AssetInventory` and await it. Show a "Preparing language model" state in the UI. Handle the case where the download is unavailable (no network on first run) by falling back to the legacy engine.
- `SpeechAnalyzer` is **on-device only**. There is no server-side path, which is exactly what this project needs.
- **Do not attach `SpeechDetector` alongside `SpeechTranscriber`**. There is a known conformance defect in the iOS 26.0 SDK where `SpeechDetector` does not conform to `SpeechModule`, and the `SpeechAnalyzer(modules:)` initializer will reject it. You do not need voice-activity detection for this app.
- Feed the analyzer from the extracted audio file. Finalize the analysis and drain all results before returning.

3.2 SFSpeechRecognizer — fallback transcription path (iOS 18–25)

```
import Speech

let recognizer = SFSpeechRecognizer(locale: Locale(identifier: "en-US"))
guard let recognizer, recognizer.isAvailable else {
    throw TranscriptionError.recognizerUnavailable
}

let request = SFSpeechURLRecognitionRequest(url: audioURL)
if recognizer.supportsOnDeviceRecognition {
    request.requiresOnDeviceRecognition = true
}
```

```
request.shouldReportPartialResults = false
request.addsPunctuation = true
request.taskHint = .dictation
```

Word timing comes from `result.bestTranscription.segments` — an array of `SFTranscriptionSegment`, each exposing:

- `substring: String`
- `timestamp: TimeInterval` — seconds from the start of the audio
- `duration: TimeInterval`
- `confidence: Float`

Convert to `CMTimeRange` with `CMTime(seconds:preferredTimescale: 600)`.

Wrap the completion-handler API in `withCheckedThrowingContinuation` so both engines expose the same `async` interface. Guard the continuation against double-resume — `recognitionTask` can fire its handler more than once if `shouldReportPartialResults` is misconfigured.

3.3 AVAssetExportSession — async export

```
let session = AVAssetExportSession(
    asset: composition,
    presetName: AVAssetExportPreset1920x1080
)!
session.videoComposition = videoComposition
session.audioMix = audioMix
session.shouldOptimizeForNetworkUse = true

// Start progress observation BEFORE awaiting the export.
let progressTask = Task {
    for await state in session.states(updateInterval: 0.1) {
        if case .exporting(let progress) = state {
            await MainActor.run { self.fractionComplete = progress.fractionComple
        }
    }
}

do {
    try await session.export(to: outputURL, as: .mp4)
    progressTask.cancel()
} catch {
```

```

        progressTask.cancel()
        throw ExportError.failed(underlying: error)
    }

```

Two rules that cause hard crashes if broken:

- **An export session is single-use.** Calling export a second time on the same instance raises `NSInternalInconsistencyException` — “Cannot call `exportAsynchronouslyWithCompletionHandler`: more than once” — and terminates the app. Construct a **fresh** `AVAssetExportSession` for every export attempt, including retries after failure.
- **Start the `states(updateInterval:)` consumption task before awaiting `export(to:as:)`.** If you start it after, the export has already finished and the stream yields nothing. Progress will sit at zero and then jump to done.

`export(to:as:)` is iOS 18+. Because the deployment target is iOS 18, do not write an `exportAsynchronously(completionHandler:)` path at all.

3.4 AVVideoCompositionCoreAnimationTool — burned-in captions and overlays

```

let renderSize = CGSize(width: 1080, height: 1920)

let parentLayer = CALayer()
let videoLayer = CALayer()
parentLayer.frame = CGRect(origin: .zero, size: renderSize)
videoLayer.frame = CGRect(origin: .zero, size: renderSize)

parentLayer.addSublayer(videoLayer)
parentLayer.addSublayer(overlayLayer) // censor images
parentLayer.addSublayer(captionLayer) // captions on top

videoComposition.renderSize = renderSize
videoComposition.frameDuration = CMTime(value: 1, timescale: 30)
videoComposition.animationTool = AVVideoCompositionCoreAnimationTool(
    postProcessingAsVideoLayer: videoLayer,
    in: parentLayer
)

```

Non-negotiable rules:

- **Every animation must set `beginTime = AVCoreAnimationBeginTimeAtZero`.** A literal `0`

is interpreted as “now” and is silently discarded during export. The result is a video where captions never appear and no error is raised. This is the single most common cause of “my captions didn’t show up.”

- Set `isRemovedOnCompletion = false` and `fillMode = .both` on every animation.
- `parentLayer.frame` and `videoLayer.frame` must exactly equal `videoComposition.renderSize`. Mismatch produces black bars, cropping, or a zoomed frame.
- Core Animation’s coordinate origin is bottom-left in this context. Y-positions computed from a top-left UIKit mental model will be vertically mirrored. Compute caption Y as `renderSize.height * (1.0 - normalizedTopOffset)` and verify visually at Gate 6.
- **This tool renders on export only.** Layers added this way will not appear in an `AVPlayerLayer`. Do not attempt to make it preview. Preview is built separately per §3.5.
- **Do not combine this tool with a custom `AVVideoCompositing`** (needed for Core Image region blur) in the same composition. They are mutually exclusive. If the user enables both burned-in captions and region blur, run two sequential export passes: pass 1 applies the blur via the custom compositor, pass 2 applies captions via the animation tool to pass 1’s output.

3.5 Preview — `AVSynchronizedLayer`

Because §3.4 will not preview, build a **second, independent copy** of the caption and overlay layer trees and attach it to an `AVSynchronizedLayer(playerItem:)` positioned above the `AVPlayerLayer`.

```
let syncLayer = AVSynchronizedLayer(playerItem: playerItem)
syncLayer.frame = playerLayer.videoRect
syncLayer.addSublayer(previewOverlayLayer)
syncLayer.addSublayer(previewCaptionLayer)
playerLayer.addSublayer(syncLayer)
```

`AVSynchronizedLayer` evaluates `CAAnimation` timings against the player item’s clock, so the same `beginTime / keyTimes` values work in both trees. Both trees must be generated from the same builder functions with only the coordinate space differing — one at `renderSize`, one at `playerLayer.videoRect.size`. Never hand-write the preview tree separately; it will drift from the export tree.

Apply the `AVAudioMix` directly to the `AVPlayerItem` so beeps are audible during preview:

```
playerItem.audioMix = audioMix
```

3.6 Asset loading — async only

`AVAsset(url:)` is deprecated. Use `AVURLAsset(url:)` .

Load every property asynchronously. Synchronous property access produces deprecation warnings and can block the calling thread for seconds on large files:

```
let asset = AVURLAsset(url: url)
let duration = try await asset.load(.duration)
let videoTracks = try await asset.loadTracks(withMediaType: .video)
guard let track = videoTracks.first else { throw ImportError.noVideoTrack }
let (transform, naturalSize, frameRate) = try await track.load(
    .preferredTransform, .naturalSize, .nominalFrameRate
)
```

3.7 Beep synthesis — AVAudioFile

Do not ship an audio asset. Generate the tone at runtime:

```
let format = AVAudioFormat(standardFormatWithSampleRate: 44100, channels: 1)!
let frameCount = AVAudioFrameCount(duration * 44100)
let buffer = AVAudioPCMBuffer(pcmFormat: format, frameCapacity: frameCount)!
buffer.frameLength = frameCount

let samples = buffer.floatChannelData![0]
let envelopeFrames = Int(0.005 * 44100) // 5ms attack/release

for i in 0..
```

```
let file = try AVAudioFile(forWriting: url, settings: format.settings)
try file.write(from: buffer)
```

The 5ms cosine envelope is required. A raw sine that starts and ends at a non-zero sample produces an audible click at every censor boundary.

PART IV — PROJECT STRUCTURE

Single Xcode project. One app target. One unit test target. No app extensions.

```
BleepKit/
├─ App/
│   ├── BleepKitApp.swift
│   ├── RootView.swift
│   └─ AppEnvironment.swift           // dependency container
├─ Models/
│   ├── WordToken.swift
│   ├── Transcript.swift
│   ├── CensorRange.swift
│   ├── CaptionStyle.swift
│   ├── CensorStyle.swift
│   ├── BeepSettings.swift
│   └─ Project.swift                 // @Model, SwiftData
├─ Services/
│   ├── Transcription/
│   │   ├── TranscriptionEngine.swift // protocol
│   │   ├── SpeechAnalyzerEngine.swift // iOS 26+
│   │   ├── LegacySpeechEngine.swift  // SFSpeechRecognizer
│   │   ├── TranscriptionService.swift // façade + engine selection
│   │   └─ AudioExtractor.swift        // video -> m4a
│   ├── Profanity/
│   │   ├── TextNormalizer.swift
│   │   ├── ProfanityMatcher.swift
│   │   ├── ProfanityList.swift       // JSON decoding
│   │   └─ profanity_en.json
│   ├── Audio/
│   │   ├── BeepGenerator.swift
│   │   ├── CensorRangeCalculator.swift // padding + merging
│   │   └─ AudioCensorBuilder.swift
│   └─ Video/
│       ├── CompositionBuilder.swift
│       └─ CaptionLayerBuilder.swift
```

```

|   |   |─ OverlayLayerBuilder.swift
|   |   |─ TextMeasurement.swift
|   |   └─ ExportPipeline.swift
|   └─ Storage/
|       |─ ProjectStore.swift
|       |─ TempFileManager.swift
|       └─ PhotoLibraryWriter.swift
|─ ViewModels/
|   |─ ImportViewModel.swift
|   |─ EditorViewModel.swift
|   └─ ExportViewModel.swift
|─ Views/
|   |─ ImportView.swift
|   |─ EditorView.swift
|   |─ PlayerView.swift           // UIViewRepresentable
|   |─ WordListView.swift
|   |─ CaptionStyleView.swift
|   |─ CensorStyleView.swift
|   |─ ExportView.swift
|   └─ Components/
|       |─ ScrubberView.swift
|       |─ ColorPickerRow.swift
|       |─ SliderRow.swift
|       └─ StateViews.swift       // loading / empty / error
|─ Support/
|   |─ CMTime+Extensions.swift
|   |─ Logger+Categories.swift
|   └─ Result+Extensions.swift
|─ Resources/
|   |─ Assets.xcassets           // bundled censor stickers
|   └─ Info.plist

```

Architecture: MVVM with protocol-injected services.

- Services are `actor` types or `Sendable` structs. They never touch UIKit or SwiftUI.
- View models are `@MainActor @Observable` classes. They own state and call services.
- Views are pure SwiftUI and hold no business logic.
- `AppEnvironment` constructs concrete services and injects them, so every service is swappable for testing.

PART V — DATA MODELS

5.1 WordToken

```
struct WordToken: Identifiable, Hashable, Sendable, Codable {
    let id: UUID
    var text: String           // as recognized; may arrive pre-masked as "f**
    var displayText: String    // what the caption renders after censor treatm
    var startSeconds: Double
    var durationSeconds: Double
    var confidence: Float
    var detectedProfane: Bool   // result of ProfanityMatcher
    var userOverride: Bool?     // nil = use detection, true = force censor, fa

    var isCensored: Bool { userOverride ?? detectedProfane }

    var range: CMTimeRange {
        CMTimeRange(
            start: CMTime(seconds: startSeconds, preferredTimescale: 600),
            duration: CMTime(seconds: durationSeconds, preferredTimescale: 600)
        )
    }
}
```

`CMTime` and `CMTimeRange` are not `Codable`. Persist seconds as `Double` and reconstruct. Timescale 600 throughout — it divides evenly into 24, 25, 30, and 60 fps.

5.2 CensorRange

```
struct CensorRange: Hashable, Sendable {
    var start: CMTime
    var end: CMTime
    var sourceTokenIDs: [UUID] // may be several after merging

    var timeRange: CMTimeRange {
        CMTimeRange(start: start, end: end)
    }
}
```

5.3 CaptionStyle

```
struct CaptionStyle: Codable, Hashable, Sendable {
    enum FontFamily: String, Codable, CaseIterable {
        case sfPro, sfProRounded, newYork
    }
}
```

```

    }

    var fontFamily: FontFamily = .sfProRounded
    var fontSize: CGFloat = 72
    var fontWeight: Int = 800 // maps to UIFont.Weight
    var fillColorHex: String = "#FFFFFF"
    var activeColorHex: String = "#FFD60A"
    var strokeWidth: CGFloat = 6
    var strokeColorHex: String = "#000000"
    var shadowOpacity: Float = 0.6
    var shadowRadius: CGFloat = 8
    var backgroundEnabled: Bool = false
    var backgroundColorHex: String = "#000000"
    var backgroundOpacity: Float = 0.5
    var backgroundCornerRadius: CGFloat = 16
    var backgroundPadding: CGFloat = 20
    var verticalPositionNormalized: CGFloat = 0.78 // 0 = top, 1 = bottom
    var maxWordsPerLine: Int = 3
    var maxSecondsPerLine: Double = 1.6
}

```

5.4 CensorStyle

```

enum CensorStyle: Codable, Hashable, Sendable {
    case asterisks // "f***"
    case blackBar // filled rounded rect over glyph bound
    case image(assetIdentifier: String) // user-picked or bundled sticker
    case emoji(String) // single emoji glyph
}

```

5.5 BeepSettings

```

struct BeepSettings: Codable, Hashable, Sendable {
    var frequencyHz: Double = 1000 // range 400...2000
    var levelDBFS: Double = -12 // range -24...-6
    var paddingSeconds: Double = 0.06 // range 0...0.2
    var rampSeconds: Double = 0.02

    var amplitude: Double { pow(10, levelDBFS / 20) }
}

```

5.6 Project (SwiftData)

```

@Model
final class Project {
    @Attribute(.unique) var id: UUID
    var createdAt: Date
    var updatedAt: Date
    var title: String
    var sourceFileName: String           // relative to sandbox Documents
    var durationSeconds: Double
    var tokensData: Data                 // JSON-encoded [WordToken]
    var captionStyleData: Data
    var censorStyleData: Data
    var beepSettingsData: Data
    var overlayEnabled: Bool
    var overlayAssetIdentifier: String?
    var overlayPositionX: Double
    var overlayPositionY: Double
    var enabledSeverities: [String]      // ["mild", "strong", "slur"]
}

```

Store the source video by **relative** filename, never an absolute URL. Sandbox container paths change between app launches and installs; a stored absolute path will be dead on next launch.

PART VI — BUILD PHASES

Phase 1 — Scaffold, Import, Permissions

Build:

- Xcode project, iOS 18 deployment target, Swift 6 language mode, portrait-only, iPhone-only.
- SwiftData container with the `Project` model.
- `TempFileManager` owning a scratch directory under `Caches` , purging it on launch.
- `ImportView` with `PhotosPicker(selection:matching: .videos)` .
- On selection: copy the asset into the app sandbox under `Documents/sources/<uuid>.mov` . Never mutate or reference the original Photos asset for processing.

- Async load of duration, natural size, preferred transform, nominal frame rate per §3.6.
- Orientation normalization: compute the transform required to render the source as portrait 1080×1920 with aspect-fill, center-cropped. Handle all four `preferredTransform` orientations.
- `AudioExtractor` : export the audio track to `Caches/scratch/<uuid>.m4a` using a fresh `AVAssetExportSession` with `AVAssetExportPresetAppleM4A` . This file feeds transcription.
- Also accept video via `.fileImporter` and via the share sheet (`CFBundleDocumentTypes` for `public.movie`).

Info.plist additions — write real, human-readable copy, not placeholder text:

- `NSSpeechRecognitionUsageDescription` — “BleepKit transcribes your video on this device to generate captions. Audio never leaves your iPhone.”
- `NSPhotoLibraryUsageDescription` — “BleepKit needs access to your videos so you can pick a Reel to caption.”
- `NSPhotoLibraryAddUsageDescription` — “BleepKit saves your finished video back to your Photo Library.”
- `UISupportedInterfaceOrientations` — portrait only.

Request authorization at point of use, never at launch.

`SFSpeechRecognizer.requestAuthorization` before transcription;
`PHPhotoLibrary.requestAuthorization(for: .addOnly)` before saving. Handle `.denied` and `.restricted` with an explanatory view and a deep-link to `UIApplication.openSettingsURLString` .

Verification Gate 1: App launches on a physical device. User picks a video. The UI displays duration, source resolution, and frame rate. A `.m4a` exists in the scratch directory and plays back. Relaunching the app does not orphan or lose the copied source file.

Phase 2 — Transcription

Build the protocol:

```
protocol TranscriptionEngine: Sendable {
    static var isAvailable: Bool { get }
    var identifier: String { get }
    func transcribe(audioURL: URL, locale: Locale) async throws -> [WordToken]
```

```
}
```

SpeechAnalyzerEngine per §3.1. Availability-gated to iOS 26+. Must:

- Verify locale support and model installation before analysis; trigger and await installation if needed, with a progress state surfaced to the UI.
- Feed the extracted `.m4a`, finalize the analysis, and drain all results.
- Map each `AttributedString` `run` to a `WordToken` using the run's `audioTimeRange`.
- Split runs containing multiple words on whitespace, subdividing the time range proportionally by character count.

LegacySpeechEngine per §3.2. Must:

- Wrap the completion-handler API in a continuation, guarded against double-resume.
- Map each `SFTranscriptionSegment` to a `WordToken`.

TranscriptionService selects the engine: `SpeechAnalyzerEngine` when `#available(iOS 26, *)` **and** the model is installed; otherwise `LegacySpeechEngine`. Surface the active engine identifier in a small debug label in the editor so behavior differences are diagnosable in the field.

Caching: persist the resulting `[WordToken]` to the `Project` record. Reopening a project must never re-transcribe.

Cancellation: a cancelled `Task` must tear down the analyzer or recognition task and delete partial scratch files. Verify with a cancel button during a long transcription.

Verification Gate 2: Transcribe a 30-second clip. Log every token with `text`, `startSeconds`, and `durationSeconds`. Manually spot-check three words against the video timeline — timings must land within roughly 150ms. If any token has a zero or missing duration, the `attributeOptions` configuration or the segment parsing is wrong. Fix it before proceeding. Everything downstream depends on these numbers being correct.

Phase 3 — Profanity Detection

`profanity_en.json`

Bundled in the app. No network fetch, ever. Schema:

```
{
```

```

"version": 1,
"terms": [
  { "term": "fuck",      "severity": "strong", "variants": ["fucking", "fucked",
  { "term": "shit",      "severity": "strong", "variants": ["shitting", "shitty",
  { "term": "bitch",     "severity": "strong", "variants": ["bitches", "bitching"
  { "term": "cunt",      "severity": "strong", "variants": ["cunts"] },
  { "term": "dick",      "severity": "strong", "variants": ["dicks", "dickhead"]
  { "term": "cock",      "severity": "strong", "variants": ["cocks"] },
  { "term": "pussy",     "severity": "strong", "variants": ["pussies"] },
  { "term": "asshole",   "severity": "strong", "variants": ["assholes"] },
  { "term": "bastard",   "severity": "strong", "variants": ["bastards"] },
  { "term": "damn",      "severity": "mild",   "variants": ["damned", "damnit", "
  { "term": "hell",      "severity": "mild",   "variants": [] },
  { "term": "ass",       "severity": "mild",   "variants": ["asses"] },
  { "term": "crap",      "severity": "mild",   "variants": ["crappy", "craps"] },
  { "term": "piss",      "severity": "mild",   "variants": ["pissed", "pissing"]
  { "term": "bloody",    "severity": "mild",   "variants": [] }
],
"allowlist": [
  "assassin", "assassinate", "assess", "assessment", "asset", "assets",
  "assign", "assignment", "assist", "assistant", "associate", "assume",
  "class", "classic", "glass", "grass", "pass", "passage", "mass", "bass",
  "analysis", "analyst", "analyze", "cockpit", "cocktail", "peacock",
  "shiitake", "scunthorpe", "penistone", "sussex", "middlesex", "titan",
  "document", "matchstick", "dickens", "hello", "shell", "shelter"
]
}

```

Expand the `terms` array to a complete common-English profanity list. For the `slur` severity tier, populate from a maintained public list rather than enumerating inline — mark the tier as enabled by default and non-toggleable, since creators almost never want those uncensored.

TextNormalizer

Applied to every token before matching:

1. Lowercase
2. Unicode normalize to NFKD, strip combining marks
3. Collapse runs of 3+ identical characters to one — `fuuuuuck` → `fuck`, `shiiit` → `shit`
4. Leetspeak substitution: `@` → `a`, `4` → `a`, `0` → `o`, `1` → `i`, `!` → `i`, `|` → `i`, `3` → `e`, `$` → `s`, `5` → `s`, `7` → `t`, `+` → `t`
5. Strip all non-alphanumerics **except** the asterisk — the asterisk is meaningful, see below

Normalization is pure and synchronous. Unit-test it independently.

`ProfanityMatcher`

Rules:

- **Whole-word boundary matching only.** Substring matching creates the Scunthorpe problem and will censor “class” and “assassin.” The allowlist is a second line of defense, not the primary mechanism.
- Check the normalized token against `term` and every entry in `variants`.
- Handle inflections not covered by explicit variants: strip trailing `-s`, `-es`, `-ing`, `-ed`, `-er`, `-in'` and retry the match.
- **Treat any token matching `\w?\*{2,}\w?` as a positive hit.** Apple’s speech recognizers are known to return profanity pre-masked — you will receive `f***` rather than the literal word, and there is no public API to disable this. The masked token still carries correct timing, so the mask itself is a reliable detection signal.
 - Confidence: **high** that `SFSpeechRecognizer` masks profanity. **Unverified** for `SpeechTranscriber` on iOS 26. This is precisely why the matcher must handle both the literal word and the mask. Do not assume either behavior.
- Multi-word phrase support: check consecutive token windows of length 2 and 3 against terms containing spaces.
- Severity filtering: a term only counts as a hit if its severity tier is currently enabled by the user.

User controls

`WordListView` — a scrollable list of every token showing text, timestamp, and censor state. Tapping a word seeks the player to it. A toggle on each row sets `userOverride` to `true` (force censor) or `false` (force allow), or clears it back to `nil` (use detection). Overrides persist to `SwiftData` and survive re-transcription and app relaunch.

Verification Gate 3: Unit tests, all green:

- `assassin`, `class`, `analysis`, `cockpit`, `shiitake`, `passage` **are not** flagged
- `f*ck`, `fuuuck`, `sh1t`, `@ss`, `FUCK`, `Fucking` **are** flagged
- `f***` and `s***` **are** flagged
- A `mild` term is not flagged when the mild tier is disabled

- A `userOverride` of `false` on a detected term yields `isCensored == false`
-

Phase 4 — Censor Range Calculation

`CensorRangeCalculator` is a pure function and the shared source of truth for both audio and video censoring. Both layers consume its output, which is what keeps them frame-aligned.

```
func censorRanges(  
    tokens: [WordToken],  
    padding: Double,  
    assetDuration: CMTime  
) -> [CensorRange]
```

Algorithm:

1. Filter to tokens where `isCensored == true`.
2. Expand each token's range by `±padding` seconds.
3. Clamp to `[.zero, assetDuration]`.
4. Sort by start time.
5. **Merge overlapping and adjacent ranges.** Two ranges merge if `next.start <= current.end`. Accumulate `sourceTokenIDs` across merges.
6. Return the merged, sorted array.

The padding exists because speech-recognition word boundaries drift relative to the actual audio. Without it, the leading or trailing phoneme of the curse leaks through the beep. This is the single most common failure mode in this product category. Default 60ms, user-adjustable 0–200ms.

Merging is equally load-bearing. Two curses spoken back to back produce overlapping padded ranges; if you do not merge them, you insert two beep instances that overlap, sum to clipping, and produce an audible artifact.

Verification Gate 4: Unit tests:

- Two tokens 40ms apart with 60ms padding merge into exactly one range
- Two tokens 500ms apart with 60ms padding remain two ranges
- A token at time zero clamps its start to `.zero` rather than going negative

- A token at the end clamps to `assetDuration`
 - `sourceTokenIDs` correctly accumulates across a 3-way merge
-

Phase 5 — Audio Censoring

`BeepGenerator` per §3.7. Generates a `.caf` in the scratch directory. Caches by `(frequency, level, duration)` so identical beeps are synthesized once per session.

`AudioCensorBuilder`:

1. Build an `AVMutableComposition`.
2. **Track A** — insert the source video's audio track for the full duration.
3. **Track B** — for each `CensorRange`, generate a beep of exactly that range's duration and insert it at that range's start time.
4. Build `AVMutableAudioMixInputParameters` for Track A. For each `CensorRange`:
 - `setVolumeRamp(fromStartVolume: 1.0, toEndVolume: 0.0, timeRange:) over rampSeconds` starting at `range.start`
 - `setVolume(0.0, at: range.start + rampSeconds)`
 - `setVolumeRamp(fromStartVolume: 0.0, toEndVolume: 1.0, timeRange:) over rampSeconds` ending at `range.end`
5. Track B needs no mix parameters — the beep is already at the target level from synthesis.
6. Return the composition and the `AVMutableAudioMix`.

Edge cases to handle explicitly:

- Source video with no audio track — skip audio censoring entirely, still produce captions
- A censor range shorter than $2 \times \text{rampSeconds}$ — shorten the ramps proportionally rather than producing overlapping ramps
- A censor range starting at exactly `.zero` — no lead-in ramp, start muted

Verification Gate 5: Export audio-only from the composition. Listen on headphones. Every profane word is fully inaudible. The beep covers each one completely with no leading or trailing leak. There is no click, pop, or discontinuity at any ramp boundary. Two back-to-back curses produce one continuous beep, not two overlapping tones.

Phase 6 — Captions

Line grouping

Group tokens into caption lines: **max 3 words OR max 1.6 seconds**, whichever limit is reached first. Both values come from `CaptionStyle` and are user-adjustable. A line's time range spans from its first token's start to its last token's end. Insert no gaps — if there is dead air between lines, extend the previous line's display until the next line begins, so captions never flicker off between phrases.

Text measurement

`TextMeasurement` computes exact glyph bounds using `CTLineGetTypographicBounds` and `CTLineGetBoundsWithOptions`. Black bars and censor images must align to the **actual rendered glyph rect**, not an estimate from character count. An estimate will be visibly wrong at large font sizes.

Layer construction

`CaptionLayerBuilder` returns a `CALayer` sized to the target coordinate space (passed in, so the same builder serves both export at `renderSize` and preview at `videoRect.size`).

For each caption line, build a `CATextLayer` (or a composed layer of per-word `CATextLayer`s for karaoke highlighting) with:

- `contentsScale = UIScreen.main.scale` for export sharpness — a missing `contentsScale` produces blurry text
- `opacity` driven by a `CAKeyframeAnimation` with `calculationMode = .discrete`, keyed to the line's time range
- `beginTime = AVCoreAnimationBeginTimeAtZero`, `isRemovedOnCompletion = false`, `fillMode = .both`

Karaoke highlight: the active word renders in `activeColorHex`, inactive words in `fillColorHex`. Drive with a `CAKeyframeAnimation` on `foregroundColor`, `calculationMode = .discrete`, with `keyTimes` normalized against the animation's total duration.

Style controls (all system fonts only)

Font family (SF Pro, SF Pro Rounded, New York), size, weight, fill color, active color, stroke width and color, drop shadow opacity and radius, background pill with color/opacity/corner radius/padding, vertical position.

Vertical position defaults to 0.78 down the frame to clear Instagram’s own UI chrome. Clamp so captions never render below 88% or above 15% of frame height.

Caption censor treatments

Style	Rendering
<code>asterisks</code>	First character, then <code>*</code> repeated for the remaining length — <code>f***</code>
<code>blackBar</code>	A filled rounded rect covering the measured glyph bounds, inset by 2pt
<code>image</code>	User-picked image scaled aspect-fit into the glyph bounds, centered
<code>emoji</code>	A single emoji glyph scaled to the glyph bounds’ height

The censored word still occupies its correct position in the line — replace the rendering, not the layout.

Verification Gate 6: Export a 10-second clip with captions burned in. Confirm:

- Captions appear from frame 1, not partway through
- The highlight advances word by word in sync with the audio
- Censored words show the selected treatment, correctly positioned over the word
- Nothing is clipped off any frame edge
- Text is sharp, not blurry

If captions do not appear at all, `beginTime` is wrong — see §3.4. If they are vertically mirrored, the coordinate origin is wrong — see §3.4.

Phase 7 — Video Overlay and Preview

`OverlayLayerBuilder`

A layer positioned between `videoLayer` and the caption layer.

- Source image from `PhotosPicker` or a bundled sticker set in `Assets.xcassets`: black bar, blur tile, star burst, emoji sheet. Design these as simple vector-friendly PNGs at 3x.
- Placement: drag-to-position with a pinnable anchor, or “follow caption” mode which tracks the censored word’s on-screen position.

- Visibility from a `CAKeyframeAnimation` on `opacity`, `calculationMode = .discrete`, keyed to the **same merged `CensorRange` array** produced in Phase 4. Reusing those ranges is what guarantees the overlay and the beep start and end on the same frame.

Optional: region blur (second export pass)

If the user enables blur instead of an image overlay, implement a custom

`AVVideoCompositing` that applies `CIGaussianBlur` restricted to a user-defined rect for the duration of each censor range. Run this as **pass 1**, then run the caption animation tool as **pass 2** on pass 1's output. Do not attempt both in one composition — see §3.4.

Preview

`PlayerView` as a `UIViewRepresentable` containing an `AVPlayerLayer` plus an `AVSynchronizedLayer(playerItem:)` per §3.5, carrying preview copies of the overlay and caption trees generated by the same builders at the preview coordinate scale.

Apply the audio mix to the `AVPlayerItem` so beeps are audible.

Editor UI: scrubber with current-time readout, play/pause, frame-step buttons, and the tappable `WordListView` that seeks to a token's start time.

Verification Gate 7: Preview and export show identical caption text, identical overlay placement, and identical timing. Scrub to three censored words in preview, note the visual state, export, and compare the same three timestamps in the exported file. Any divergence means the two layer trees are being built differently — reconcile the builders before proceeding.

Phase 8 — Export and Delivery

`ExportPipeline` per §3.3.

- `renderSize` `1080×1920`. `frameDuration` matched to the source's nominal frame rate, clamped to `24/25/30/60`.
- Preset `AVAssetExportPreset1920x1080`. Output type `.mp4`.
`shouldOptimizeForNetworkUse = true`.
- Fresh `AVAssetExportSession` per attempt, including retries.
- Progress task started before awaiting the export.
- On success, `PhotoLibraryWriter` saves via `PHPhotoLibrary.shared().performChanges`

with `PHAssetCreationRequest`, into a "BleepKit" album created on first export. Also offer a share sheet with the file URL.

- On failure, surface the actual `AVAssetExportSession.error.localizedDescription` in the UI. Never fail silently, never show a generic "Something went wrong."
- Delete scratch artifacts on both success and failure.
- Cancellation: cancel the session, cancel the progress task, delete the partial output file, restore the editor state.

Verification Gate 8: Full end-to-end on a physical device. Import a real Reel containing profanity, transcribe, review and adjust flags, style captions, export, and upload to Instagram. The uploaded video plays with correct captions, correct beeps, and correct overlays, with no re-encode artifacts and no audio/video drift at the end of a 60-second clip.

Phase 9 — Hardening

- **Long-video handling.** Test a 3-minute clip. Confirm memory stays flat — no full-video load into RAM. Use `AVAssetReader` streaming for anything that touches samples.
- **Interruption handling.** Backgrounding mid-export, incoming phone call mid-preview, low-storage during export.
- **Empty and degenerate inputs.** Video with no audio track. Video with audio but no detected speech. Video where every word is profane. Video 1 second long.
- **Locale handling.** Non-English source audio: detect the mismatch, inform the user that profanity detection is English-only, and still produce captions.
- **State restoration.** Force-quit mid-edit, relaunch, confirm the project reopens with all tokens, overrides, and style settings intact.
- **Accessibility.** VoiceOver labels on every control. Dynamic Type in all UI chrome (not in the burned-in captions, which are fixed by design). Minimum 44pt tap targets.
- **Instruments pass.** Time Profiler during transcription and export. Allocations during a 3-minute export. Fix anything that blocks the main thread for more than 16ms.

Verification Gate 9: All of the above pass on a physical device. No memory growth across five consecutive exports.

PART VII — NON-FUNCTIONAL REQUIREMENTS

- **Concurrency.** Swift 6 strict mode, zero data-race diagnostics. Services are `actor` or `Sendable`. View models are `@MainActor`. All async work uses structured concurrency — `async/await` and `TaskGroup`. The only completion-handler code in the project is the continuation wrapper inside `LegacySpeechEngine`.
 - **Main thread.** No blocking work. No synchronous `AVAsset` property access. No file I/O.
 - **Memory.** Never load a full video into memory. Stream via `AVAssetReader` where samples must be touched.
 - **Temp files.** `TempFileManager` owns every scratch file. Purge on app launch and on project close. Never leak into `Documents`.
 - **State.** Every async surface has explicit loading, empty, and error states in the UI. No spinner without a cancel button.
 - **Logging.** `OSLog` with per-subsystem categories: `transcription`, `profanity`, `audio`, `video`, `export`. No `print`.
 - **Safety.** No force unwraps outside `#if DEBUG` assertions. No `try!`. No implicitly unwrapped optionals.
 - **Documentation.** Doc comments on every public type, method, and non-obvious property.
-

PART VIII — DELIVERABLES

1. Complete file tree matching Part IV.
2. Every Swift file, fully implemented and compiling with zero warnings.
3. `profanity_en.json`, populated per Phase 3.
4. `Info.plist` with real usage-description strings.
5. Unit test target covering `TextNormalizer`, `ProfanityMatcher`, `CensorRangeCalculator`, and `WordToken` codable round-tripping.
6. `SETUP.md` — signing with a personal Apple Developer certificate, deployment target configuration, running on a physical device, the 7-day provisioning expiry for free accounts, and first-run language-model download behavior on iOS 26.

7. `RISKS.md` — for each failure mode in Part IX, the specific file and function that mitigates it.

PART IX — KNOWN FAILURE MODES

Every one of these must be explicitly addressed in code, and documented in `RISKS.md`.

#	Failure mode	Required mitigation
1	Speech timestamps drift from true word boundaries; the curse leaks past the beep	<code>CensorRangeCalculator</code> applies $\pm 60\text{ms}$ configurable padding before merging
2	Recognizer returns profanity pre-masked as <code>f***</code>	<code>ProfanityMatcher</code> treats <code>\w?*</code> <code>{2,}\w?</code> as a positive hit; timing remains valid
3	Adjacent curses produce stacked, clipping beeps	Overlapping padded ranges are merged before beep insertion
4	<code>AVVideoCompositionCoreAnimationTool</code> does not render in <code>AVPlayer</code>	Separate <code>AVSynchronizedLayer</code> tree for preview, built by the same builders
5	Animations with <code>beginTime = 0</code> are silently dropped on export	<code>AVCoreAnimationBeginTimeAtZero</code> on every animation, asserted in a debug pre-export check
6	Export session reused after completion raises <code>NSInternalInconsistencyException</code>	<code>ExportPipeline</code> constructs a fresh session per attempt, including retries
7	Progress observation started after <code>export(to:as:)</code> yields nothing	<code>Progress Task</code> started before the <code>await</code>
8	Core Animation's bottom-left origin mirrors caption Y positions	Y computed as <code>renderSize.height * (1 - normalizedOffset)</code> , verified visually at Gate 6
9	Substring matching censors "class" and "assassin"	Whole-word boundary matching plus a curated allowlist
10	Absolute sandbox paths break on relaunch	Sources stored and resolved by relative filename

11	Blurry burned-in text	<code>contentsScale</code> set on every <code>CATextLayer</code>
12	iOS 26 language model not yet downloaded on first run	Installation check with progress UI, and fallback to <code>LegacySpeechEngine</code>

Begin Phase 1 now. Stop at Verification Gate 1 and report before continuing.